Module 3: Placement & Routing

❖ What is the placement? What is the requirement of placement?

(A) Placement is the process of placing the cells, or searching appropriate location within the chip floor plan for each cell in the netlist. Placement and routing is an interconnecting stage in the design of PCB, integrated circuits, and FPGAs. Placement is a process that decides where to place all electronic components, circuitry, and logic elements in a generally limited amount of space. Then routing process decides the routes of all the wires needed to connect the placed components.

Placement does not just place the standard cell available in the synthesized netlist, it also optimizes the design.

A poor placement requires large area & results in performance degradation.

Objective of Placement in FPGA:

- Minimize the required wiring (congestion)
- Balance the wiring density across the FPGA
- Maximize circuit speed
- Minimize power dissipation
- Minimize crosstalk between signals & net delays

3 Major classes of placers:

- Min-cut (partitioning-based) placers
- Analytic placers
- Simulated annealing based placers

FPGA-Specifific Placement Issues

- The number of routing tracks in routing channels are fifixed on a FPGA.
- A necessary condition for any feasible placement solution is the *channel density* in every channel cannot exceed the number of routing tracks available in the channel.
- In order to calculate the channel density accurately, some SA-based placement algorithms also perform *global routing* iteratively for every move.

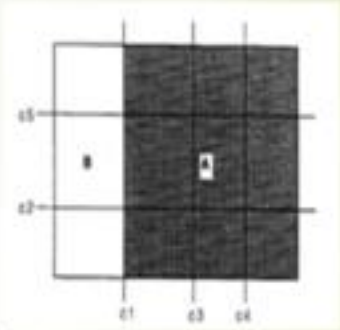
- A FPGA contains routing tracks of various lengths.
- Simple interconnection delay estimation model based on net length or fanout are not accurate enough for use in timing-driven FPGA placement algorithms.
- Fast and accurate interconnection delay computation methods are needed for timing driven FPGA placement.

❖ Discuss Mincut-based Placement

The given circuit is repeatedly partitioned into two subcircuits. Meanwhile, the chip area is partitioned alternately in the horizontal and vertical directions into subsections. Each subcircuit is assigned to a subsection. The process is repeated until each subcircuit consists of a single logic block and has a unique location on the chip area. During partitioning, the number of nets that are cut by the partition is usually minimized based on the intuition that densely connected subcircuits should be placed closely.

Partition Methods

- A commonly used partition approach is the iterative improvement approach that makes local changes to an initial partition.
- Kernighan and Lin (K-L) proposed a graph bisectioning algorithm which starts with a random initial partition and then uses pairwise swapping of vertices between partitions, until no improvement is possible.
- A simple *greedy algorithm* will accept a move only if it provides immediate benefifit, but such shortsightedness often leads to a local minimum from which the algorithm cannot escape.
- The K-L algorithm was designed to be able to escape from local minima.



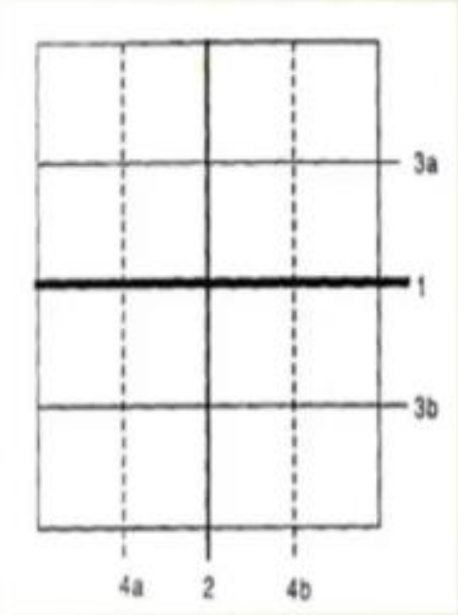
Types of Min –cut algorithm

1.Rein-Cut Placement Algorithm:

i. Begin by cutting a series of cut lines around the entire chip.
ii. Allow the chip to be partitioned into two blocks using the first cut line. Additionally, divide the circuit into two sub-circuits to reduce the net cut. Partition all of the blocks intersected by the second cut line, and then partition the circuit in the same way.
iii. Carry on in this manner with all cut lines.
iv. We must partition blocks A and B generated by c1 simultaneously while processing cut line C2.

Quadrature Placement Procedure:

I. The partitioning procedure is carried out breadth-first, with alternate vertical and horizontal cuts in this algorithm.
II. A region is divided into two equal subregions with each cut. Where there is a high routing density in the middle, this approach is appropriate.
III. The congestion in the middle is minimized by first cutting through the center and minimizing the net cut.
IV. This is the most famous cut line sequence for min-cut algorithms .

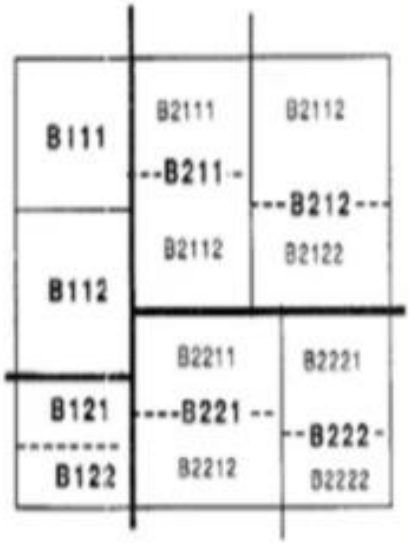


Block-Oriented Min-Cut Placement Algorithm:

I. We choose a cut line to divide the chip into two regions in this algorithm.

II. Then, for each region, we choose a separate cut line and partition the regions further.

III. This procedure is repeated until each block has just one slot.

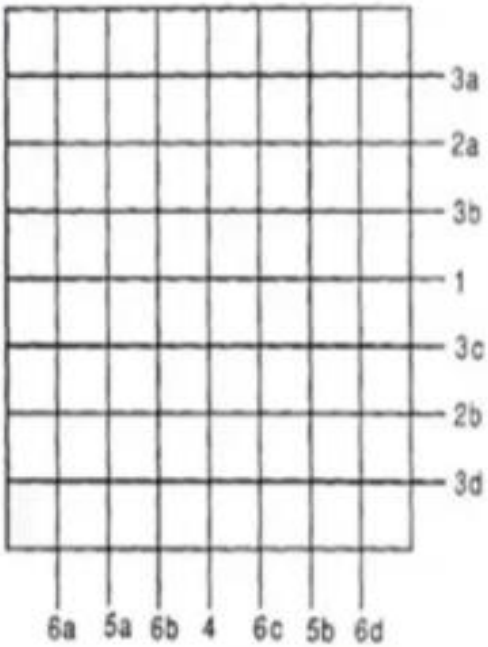


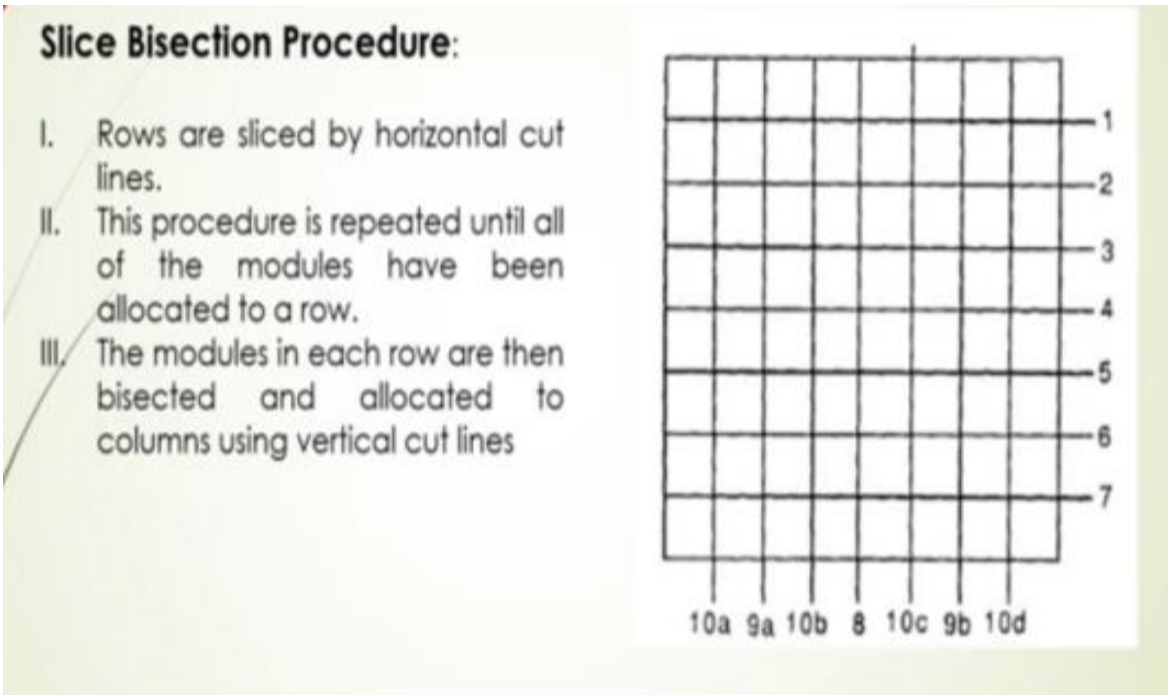
Bisection Placement Procedure:

I. The chip is bisected (divided into two equal subregions) by horizontal cut lines until each subregion consists of one row in this process.

II. Each element is assigned to a row in this method, but its location is not fixed.

III. Then each row is bisected again and again until each subregion has just one space





❖ Explain briefly Routing Technology

Routing technology in FPGA (Field-Programmable Gate Array) plays a critical role in determining how signals travel between the configurable logic elements (CLBs), interconnect resources, and I/O pins within the FPGA device. It is an essential aspect of FPGA architecture, as it directly impacts the performance, flexibility, and efficiency of FPGA designs.

Here are some key points to consider when discussing routing technology in FPGAs:

- **Interconnect Structure:** FPGAs consist of a dense grid of CLBs that are interconnected by a network of wires or routing resources. These resources allow signals to be routed between different CLBs and I/O pins. The interconnect structure can vary between FPGA families but typically includes global and local routing resources.
- **Global Routing:** Global routing resources are used to connect signals over long distances across the FPGA device. They typically include horizontal and vertical channels that span the entire FPGA fabric. These channels are used for critical paths and high-speed signals.
- **Local Routing:** Local routing resources are used to connect signals within a small region of the FPGA, such as a cluster of CLBs. These resources allow for more flexible and efficient routing of signals within specific areas of the device.
- **Switch Matrix:** The routing technology often includes switch matrices, which are configurable switches that allow designers to establish connections between various routing resources and CLBs. These switch matrices can be programmed to create custom signal paths.
- **Routing Flexibility:** FPGAs offer a high degree of routing flexibility, allowing designers to create custom interconnections between CLBs and I/O pins. This flexibility is one of the key advantages of FPGAs, as it enables the implementation of a wide range of digital logic functions.

- **Routing Constraints:** Designers must consider routing constraints when creating FPGA designs. These constraints include factors like signal propagation delays, clock skew, and the availability of routing resources. Careful consideration of these constraints is necessary to ensure that the design meets performance and timing requirements.
- **Clock Distribution:** Routing technology also plays a crucial role in distributing clock signals throughout the FPGA. Clock networks are designed to minimize clock skew and ensure that clock signals reach all parts of the device efficiently.
- **Timing Analysis:** FPGA design tools perform extensive timing analysis to ensure that signals can be routed within the specified clock frequencies. Timing constraints are used to guide the routing process and ensure that the design meets its performance goals.
- **Routing Algorithms:** FPGA vendors employ various routing algorithms to determine the most efficient path for each signal. These algorithms consider factors such as wire congestion, signal fanout, and resource availability.

In summary, routing technology in FPGAs is a fundamental aspect of their architecture that enables the flexible and efficient connection of configurable logic elements and I/O pins. Understanding the intricacies of routing and how to work within routing constraints is essential for FPGA designers to create high-performance and reliable digital designs.

❖ **Discuss general strategies for Routing FPGAs**

5.2 General Strategy for Routing In FPGAs

Because of the complexity involved, the solution of large routing problems, such as those encountered in FPGAs, usually requires a "divide and conquer" strategy. This approach is extolled in the three-step process described in [Loren89]:

1. Partition the routing resources into routing areas that are appropriate for both the device to be routed and the routing algorithms to be employed.
2. Use a *global router* to assign each net to a subset of the routing areas. The global router does not choose specific wire segments and routing switches for each connection, but rather it creates a new set of restricted routing problems.
3. Use a *detailed router* to select specific wire segments and routing switches for each connection, within the restrictions set by the global router.

The advantage of this approach is that each of the routing tools can more effectively solve a smaller part of the routing problem. More specifically, since the global router need not be concerned with allocating wire segments or routing switches, it can concentrate on more global issues, like balancing the usage of the routing channels. Similarly, with the reduced number of detailed routing alternatives that are available for each connection because of the restrictions introduced by the global router, the detailed router can focus on the problem of achieving connectivity.

This strategy has been adopted for routing in both types of FPGAs that are discussed in this chapter. The global router first selects routing channels for each connection. Then, within the constraints imposed by the global router, the detailed router implements each connection by choosing specific wire segments and routing switches. It will be apparent that the global routing issues are similar in both row-based and symmetrical FPGAs, but the detailed routing problems warrant substantially different algorithms.

- **Discuss routing for Row based**

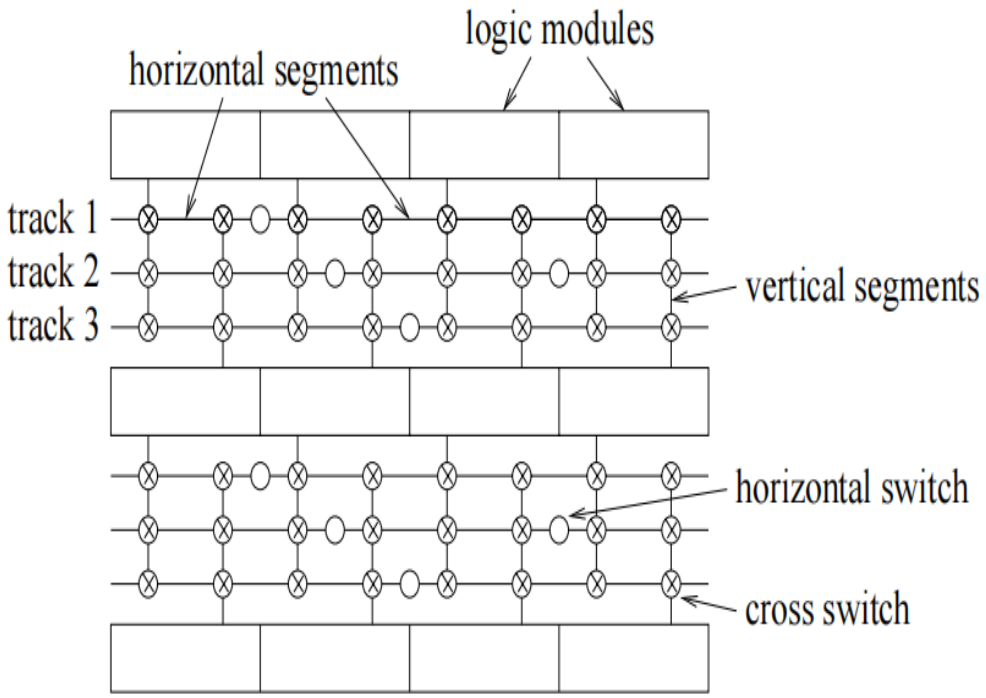
5.3 Routing for Row-Based FPGAs

As depicted earlier, in Figure 2.1, a row-based FPGA consists of rows of logic blocks that are separated by horizontal routing channels. This section describes routing algorithms that have been developed specifically for this class of FPGAs [Green90].

An FPGA based on the Actel architecture is illustrated in Figure 5.2. As shown, the routing channels consist of horizontal wire segments of various lengths, separated by routing switches. Adjacent wire segments can be joined together, allowing longer segments to be formed where necessary. Dedicated vertical segments are attached to the logic block pins and can be connected via routing switches to any horizontal wire segments that they cross. There are also other vertical segments that serve as "feed-throughs" from one routing channel to another. Note that Figure 5.2 shows only the dedicated vertical segments for the logic block marked by "*". Also, only three vertical feed-throughs are shown to avoid cluttering of the figure.

The connectivity illustrated in Figure 5.2, specifically the fact that all horizontal and vertical wire segments that cross can be connected, allows the routing problem to be partitioned into individual channels. This resembles classic channel routing in any row-based architecture, except that in classic channel routing wire segments can be placed freely wherever they are needed, whereas the wire segments in an FPGA are fixed in place before routing is performed. It is still possible to use classic routing algorithms for some special cases [Green90], but in general a new approach is required.

As indicated in Section 5.2, the first step in the routing process is global routing. This entails first dividing each multi-pin net into a set of connections, and then assigning each connection to a specific routing channel. The global router can use the vertical feed-through segments for nets that span multiple channels. Since global routing in this context is very similar to that in other row-based technologies, such as standard cells, existing algorithms can be used. Good global routing techniques have been widely published for those technologies [Loren89] [Sech87] [Cong88] [Rose90a], so they will not be described here.



A row-based routing architecture. The crossings between vertical and horizontal routing segments are fully populated with switches.

- Routing channels in row-based architectures are segmented.
- A routing track in the segmented channel is divided into several segments of various lengths with switches between the adjacent routing segments.
- In the row-based architecture, crossings between vertical and horizontal routing segments are fully populated with switches so that any vertical routing segments can be connected to any crossing horizontal routing segment where necessary.
- So detailed routing in row-based architecture is reduced to the *segmented channel routing problem*.
- Because switches can introduce signifificant delay due to the relatively high fuse re

sistance, the number of switches allowed for completing a net connection is usually restricted in order to achieve high circuit performance.

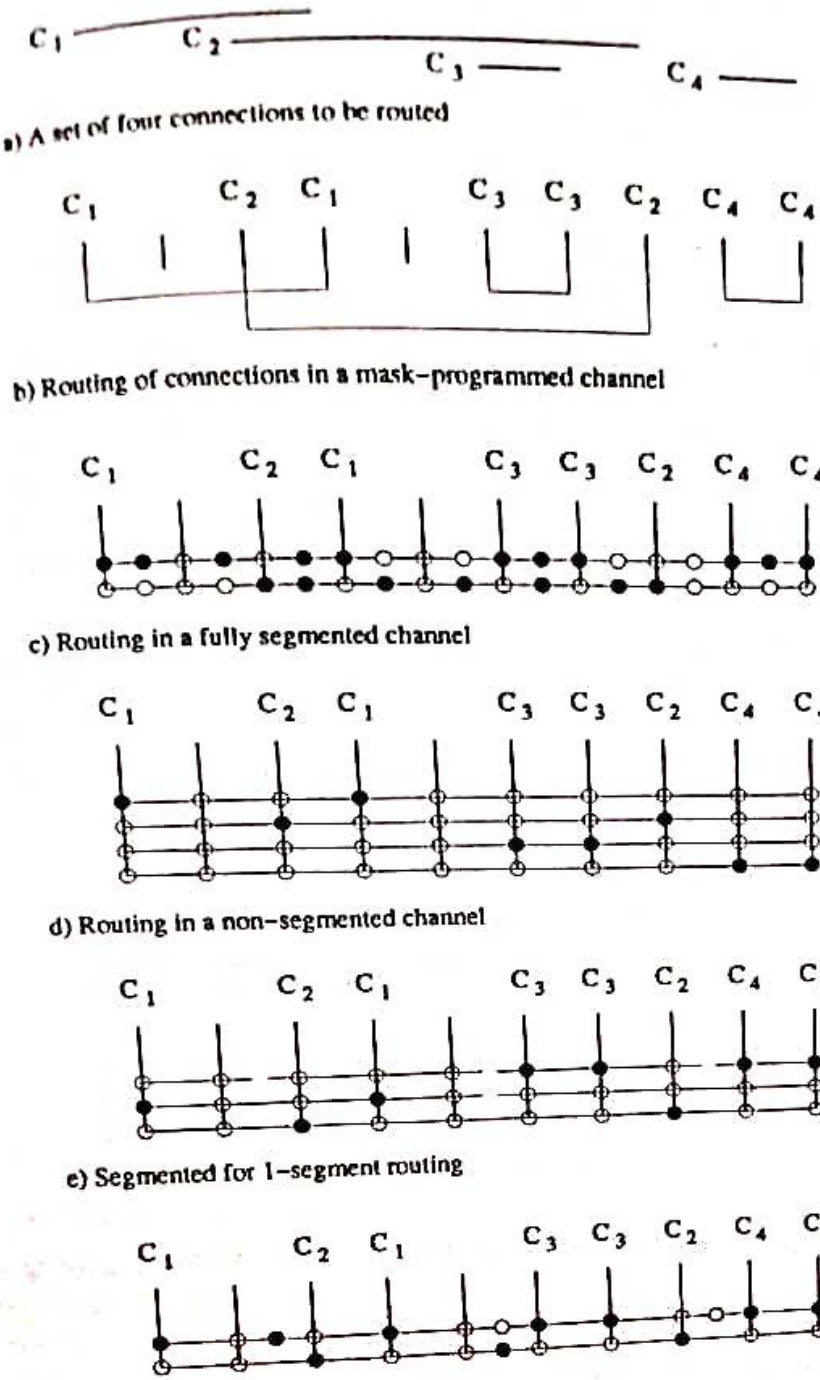
- *K-segment channel routing* requires that the maximum number of segments used for routing any net connection be limited to K
- Effificient heuristic algorithms are used to solve the general segmented channel rout ing problem.

❖ Explain segmented Channel routing

5.3.1 Introduction to Segmented Channel Routing

After global routing, each channel can be considered as a separate *detailed* routing problem. A channel will contain some number of connections, each one involving logic block pins or vertical feed-throughs. The task of the detailed routing algorithm is to allocate wire segments for each connection in a way that allows all connections to be completed. In addition, it may be necessary to minimize the routing delays of the connections. This can be accomplished by limiting the total number and length of segments used by a connection. The appropriate algorithm depends on the segmentation of the tracks and the requirements for minimizing delays, as discussed below.

Figure 5.3a shows an example of a routing problem in which the connections called C_1 to C_4 are to be routed. Figure 5.3b indicates how the connections might be routed in a mask-programmed channel, where there is complete freedom for placing the wire segments. The channel is divided into columns, as shown by the vertical segments in the figure. Some columns represent logic block pins and others are vertical feed-throughs. As the column labels in the figure indicate, each of C_1 to C_4 specifies two columns



a) A set of four connections to be routed

b) Routing of connections in a mask–programmed channel

c) Routing in a fully segmented channel

d) Routing in a non–segmented channel

e) Segmented for 1–segment routing

f) Segmented for 2–segment routing

Figure 5.3 - *Examples of Segmented Channels.*

that are to be connected. Figures 5.3c - 5.3f present several different scenarios for a segmented FPGA channel, and suggest routing solutions for each segmentation. Routing switches are indicated by circles, with an ON switch drawn solid and an OFF switch drawn hollow.

Figure 5.3c depicts one extreme for the track segmentations, in which every track is fully segmented. In this case, each segment spans only one column, meaning that multiple segments are required for every connection. As shown, the four connections can be routed in this architecture with only two tracks. A routing solution can be obtained using a straight-forward approach, such as the left-edge algorithm described in [Hash71]. In this algorithm, the connections are first sorted in ascending order according to their leftmost pins. Each connection is then assigned to the first track that is available. This simple scheme is guaranteed to require a number of tracks equal to the channel density, since there are no "vertical constraints" [Loren89] (because the vertical segments can all connect to every track). The shortcoming of the fully segmented channel is that there are an excessive number of routing switches. Since the programmable switches in FPGAs always have significant resistance and parasitic capacitance, this would cause relatively large routing delays.

The opposite extreme to full segmentation is depicted in Figure 5.3d, which shows a channel in which each track contains only one segment for its entire length. In this case, the number of tracks required for routing is always equal to the number of connections. No routing algorithm is necessary since any choice of track for a connection will do. The problem with this segmentation is that excessive area is required for the large number of tracks, and each connection will be subjected to a large capacitive load due to the long segments.

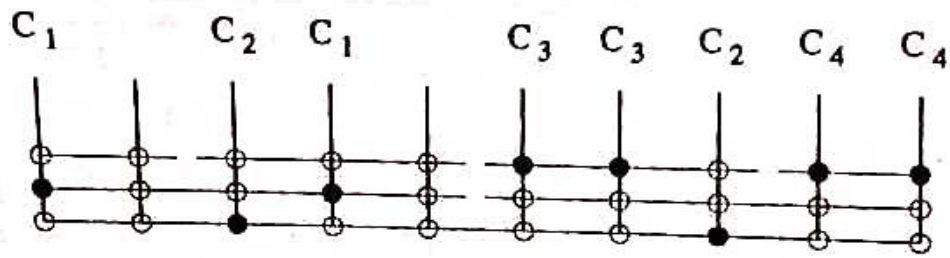
An intermediate approach to channel segmentation is illustrated in Figure 5.3e, where the tracks have segments of various lengths. Each connection must be routed in a single segment, since no switches are available where segments in the same track meet. This 1-segment problem is a special case of segmented routing and can be solved using the simple algorithm described in Section 5.3.2.

Additional flexibility in the channels can be added by allowing segments that abut to be joined by switches, as depicted in Figure 5.3f. This implies that connections can occupy more than one segment and greatly increases the complexity of the problem. A general algorithm for this class of segmented routing problem is presented in Section 5.3.3.

❖ **Explain algorithm for one segment routing**

5.3.3 An Algorithm for 1-Segment Routing

A segmented routing problem in which each connection should use only one segment is a special case that can be solved by the following greedy algorithm. Assume that there are M connections and T tracks. Assign the connections in order of increasing left ends as follows. For each connection, find the set of tracks in which the connection would occupy one segment.



e) Segmented for 1-segment routing

Eliminate any tracks in which this segment is already occupied. From the remaining tracks, assign the connection to the one whose rightmost end is furthest to the left. This simple scheme is guaranteed to find a solution for any set of connections in any segmented channel, if a solution exists. Since it is necessary to check each track for each connection, the run-time of the algorithm would be $O(MT)$. An example of a 1-segment routing problem was illustrated in Figure 5.3e. Note that for the track segmentations and connections that are shown in Figure 5.4, a valid 1-segment routing is not possible.

❖ Discuss Maze routing

(A) Maze routing in FPGA (Field-Programmable Gate Array) design is a crucial step in the process of connecting the various logic elements and signals within the FPGA's programmable fabric. It involves finding the most efficient paths through the FPGA's grid-like structure to establish interconnections while adhering to design constraints and optimizing for factors like signal timing and resource utilization. Here's a detailed discussion of maze routing in FPGA:

- **FPGA Architecture**: FPGAs consist of a grid of configurable logic elements (CLBs), interconnected by a network of routing resources. The routing resources include horizontal and vertical tracks, switch matrices, and multiplexers.

- **Nets**: In FPGA design, a "net" represents a collection of logically related signals that need to be connected. Nets can be data buses, control signals, clock distribution networks, or any other interconnection requirement within the design.
- **Routing Objective**: The primary goal of maze routing is to find the optimal paths through the FPGA fabric to connect the various components (CLBs) of the design, ensuring that signals can flow from their source to their destination efficiently.
- **Routing Algorithms**: Several routing algorithms are employed in maze routing for FPGAs, including:
  - **Lee's Algorithm**: Lee's algorithm, also known as maze expansion or wavefront propagation, starts from the source and spreads out like waves until it reaches the destination. It identifies the shortest path by following the wavefront and marking visited routing resources.
  - **A* Algorithm :** The A* algorithm is a heuristic-based search algorithm that aims to find the optimal path while considering both the cost to reach a location and a heuristic estimate of the remaining cost. It can be computationally intensive but often yields better results than Lee's algorithm.
  - **Dijkstra's Algorithm**: Dijkstra's algorithm is a graph-based algorithm that can be adapted for maze routing. It determines the shortest path by considering the cumulative cost to reach each routing point. Like A*, it can produce optimal solutions but may be slower for large FPGA designs.
- **Routing Constraints**: Designers must consider various constraints during maze routing, including:
  - **Timing Constraints**: Ensuring that signals meet timing requirements and do not exceed specified signal propagation delays.
  - **Resource Availability**: Ensuring that there are enough routing resources available to establish all the required connections.
  - **Congestion Management**: Managing congestion in routing areas to prevent overutilization of specific routing resources.
  - **Clock Distribution**: Properly routing clock signals to ensure synchronous operation within the FPGA.
- **Iterative Process**: Maze routing is often an iterative process. Design tools may perform multiple routing passes, refining the routes and addressing any congestion or timing issues as they arise.
- **Verification and Testing**: After routing is complete, designers use simulation and verification tools to ensure that the routed design functions correctly and meets timing constraints.

In summary, maze routing in FPGA design is a complex and critical process. It involves using routing algorithms to determine efficient paths through the FPGA fabric while taking into account design constraints and optimizing for various factors. The ultimate goal is to create a functional and performant FPGA design with properly routed signals and minimal resource contention.

❖ **Explain definitions for segmented channel routing**

5.3.2 Definitions for Segmented Channel Routing

Before describing the routing algorithms, this section first presents some definitions that are specific to segmented routing. A segmented routing problem consists of a set of M connections, called $C_1, \ldots, C_M$, and a set of T tracks numbered from 1 to T. Each track extends from column 1 to column N and comprises segments of various lengths. Segments that abut in the same track can be joined together by switches, which are placed between two columns. A small example of a routing problem is depicted in Figure 5.4, where M is 5, T is 3, and N is 10.

Each connection, C_i, is characterized by it leftmost column, called $left(C_i)$. It is assumed that the connections have been sorted according to their left ends, so that $left(C_i) \leq left(C_j)$ for all i less than j. A routing algorithm can *assign* a connection to a track t, in which case the segments spanned by the connection are considered *occupied*. For example, in Figure 5.4 connection C_3 would occupy the two rightmost segments if assigned to track 1, or the three rightmost segments if assigned to track 3.

A *valid routing* of a set of connections is defined as an assignment of each connection to a track such that no segment is occupied by more than one connection. A more restrictive definition is that of a *K-segment routing*, which is a valid routing that satisfies the additional requirement that no connection occupies more than K segments. As mentioned earlier, it may also be desirable to minimize the total lengths of the segments assigned to the connections, which is called *delay optimization*.

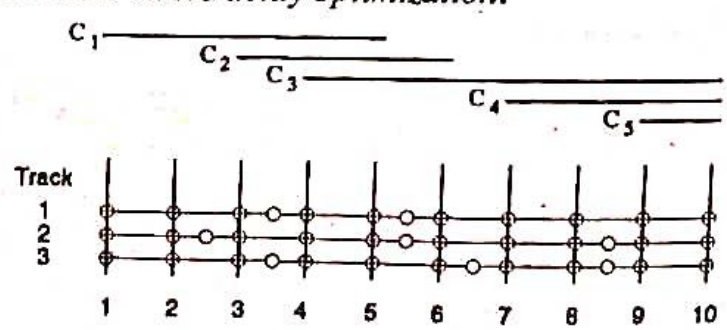


Figure 5.4 - *A Segmented Routing Problem, $M=5$, $T=3$, and $N=10$.*

❖ Explain Simulated annealing placements

- Simulated annealing (SA) mimics the annealing process used to gradually cool molten metal to produce a high-quality crystal structure.
- SA takes an existing solution and then makes successive changes in a series of random moves.
- Each move is accepted or rejected based on an *energy function*.
- In order to escape from solutions which are *local minima*, it allows "up-hill" moves that seemingly move to a less desirable solution.
- Probability of accepting an "up-hill" move is controlled by the expression $e^{\frac{-\triangle E}{T}}$ where $\triangle E$ is the resulting increase in the energy function and T is the current *temperature*.
- The temperature is slowly decreased and the system finally comes to rest at a low-energy configuration.

- An initial placement is created by assigning logic blocks of the circuit randomly to the available locations in the FPGA.
- A common cost function in wirelength-driven placement is the sum over all nets of the half-perimeter of their bounding boxes.
- A move can be the exchange of locations of two randomly selected logic blocks.
- Initially, temperature T is very high so almost all moves are accepted; it is gradually decreased as the placement is refined so that eventually the probability of accepting a move that makes the placement worst is very low.
- An advantage of simulated annealing based placer is the ease to add new optimization objectives or constraints. But a drawback is the running time required.

Algorithm: Simulated Annealing Based Placer

```
begin
S = RandomPlacement();
T = InitialTemperature();
while (ExitCriterion() == False) do
    while (InnerLoopCriterion() == False) do
        S' = GenerateViaMove(S);
        △ ← cost(S') − cost(S);
        if △ ≤ 0 then S ← S'
        if △ > 0 then S ← S' with probability e^(−△/T);
    end while;
    T = UpdateTemp();
end while;
return S
end
```

❖ **What are the iterative improvement and explain clearly**

5.4.3.3.1 Iterative Improvements

This section explains how iterations of the two phases of CGE are used to conserve memory and run-time. The iterative approach is linked to the

pruning parameters of the graph expansion phase. Setting the pruning parameters to large values allows the router to do a better job of resolving routing conflicts because it sees many alternatives for each connection. On the other hand, with large pruning parameters more memory and longer run-time are required by the algorithm. The key to this routing quality versus memory and time trade-off is the realization that most connections in an FPGA are relatively easy to route and only a small percentage of the connections pose real difficulties. This is because, in a typical routing problem, there are only a few channel segments whose densities are very close to the total number of wires in a routing channel. To exploit this property, the router starts with small pruning parameters and then increases them through successive iterations, but only for the parts of the FPGA that are difficult to route.

For the first iteration the pruning parameters are set to relatively small values, and the entire FPGA is routed. If routing conflicts leave some connections unrouted, then another iteration is required. The procedure is to erase all the routing of any connection that overlaps any part of a failed connection, and then to attempt to route those channel segments again using larger pruning parameters. Only connections that touch some segment of a channel in which a failed connection occurred are re-routed in the next iteration. Iterations are continued until all connections are routed or until further improvements are not forthcoming.

At this point it would be desirable to try different global routes for connections that are left unrouted after all iterations, but no such failure-recovery mechanism is currently implemented. This iterative approach is a minor variation of classic rip-up and re-route schemes where individual connections would be removed and re-routed to try to resolve routing conflicts. The technique employed here allows the algorithm's cost function to solve the routing problem, but conserve memory and time where the problem is not difficult and expend them only where it is required.

The specific values used for the pruning parameters in each iteration affect the total number of iterations required, but do not appreciably affect the quality of the final result. This indicates a robustness in the algorithm, because the quality of the routing does not depend on the specific values chosen for the program's parameters. For the results that are presented in Section 5.4.3.5, K and k are set to two for the first iteration. K is increased by one for each iteration until it reaches F_c, after which k is increased by one for each subsequent iteration.

Circuit	#Blocks	#Conn	Source	Type
BUSC	109	392	UTD1	Bus Cntl
DMA	224	771	UTD2	DMA Cntl
BNRE	362	1257	BNR	Logic/Data
DFSM	401	1422	UTD1	State Mach.
Z03	586	2135	Zymos	8-bit Mult

Table 5.1 - *Experimental Circuits.*

❖ **Explain the FPGA routing structures**

5.4.3.5.1 FPGA Routing Structures

Since the routability of an FPGA is determined by the topology and flexibility of its S and C blocks, those used in the tests of the algorithm are presented here. The general nature of the S block is illustrated in Figure 5.14a. Its flexibility is set by the parameter F_s, which defines the total number of connections offered to each wire segment that enters the S block. For the example shown in Figure 5.14a the wire segment at the top left of the S block can connect to six other wire segments, and so F_s is 6. Although not shown, the other wire segments are similarly connected.

Figure 5.14b illustrates the test C block. The tracks pass uninterrupted through it and are connected to logic block pins via a set of switches. The flexibility of the C block, F_c, is defined as the number of tracks that each logic block pin can connect to. For the example shown in the figure, each logic block pin can connect to 2 vertical tracks, and so F_c is 2.

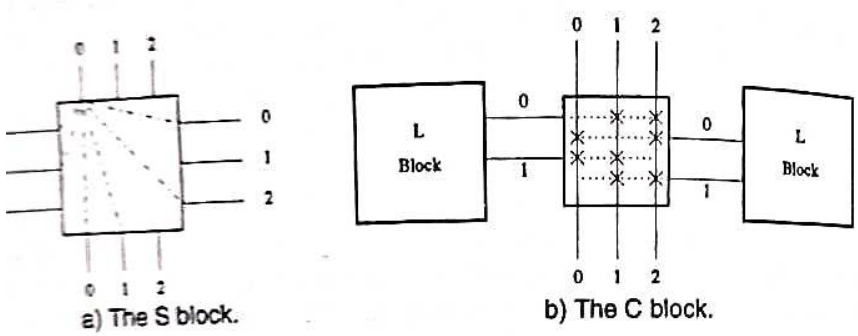
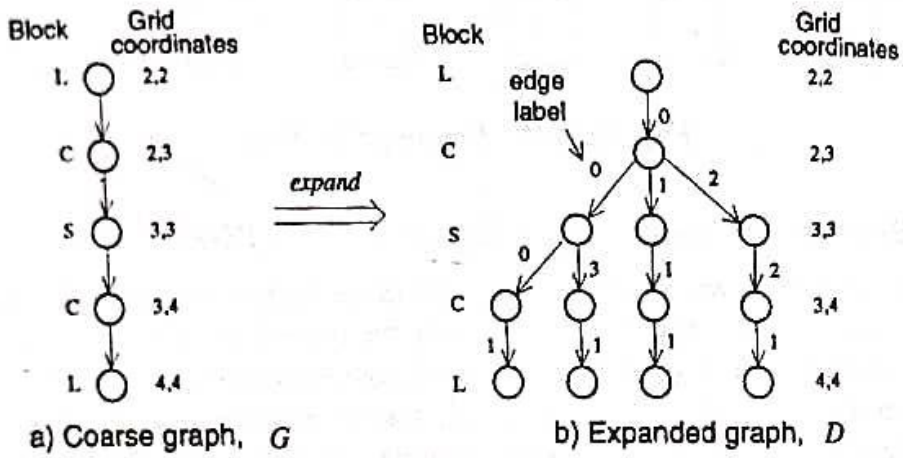


Figure 5.14 - *Definitions of S and C Block Flexibility.*

❖ **Explain the organization of CGE**



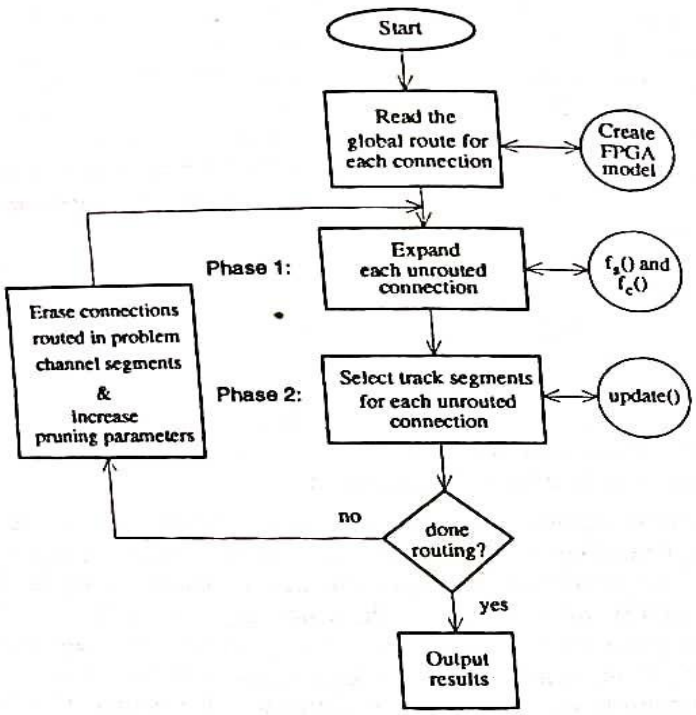


Figure 5.13 - *The Organization of CGE.*

Phase -1 Algorithm

create D and give it the same root as G. Make the immediate successor to the root of D the same as for the root of G
for each new vertex, traversing D breadth first {
expand a C vertex in D by calling $Z = f_c(e_C, n)$. e_C is the edge in D that connects to C from its predecessor. n is the required successor vertex of C (in G) and Z is the set of edges returned by $f_c(\,)$. The call to $f_c(\,)$ adds Z to D
expand an S vertex in D by calling $Z = f_s(e_S, n)$. e_S is the edge in D that connects to S from its predecessor. n is the required successor vertex of S (in G) and Z is the set of edges returned by $f_s(\,)$. The call to $f_s(\,)$ adds Z to D
}

Phase-2 Algorithm

put all the paths in the expanded graphs into the path-list
while the path-list is not empty {
if there are paths in the path-list that are known to be essential
select the essential path that has the lowest c_f cost

else if there are paths in the path-list that correspond to time-critical connections
select the critical path with the lowest c_t cost
else
select the path with the lowest c_f cost
mark the graph corresponding to the selected path as routed - remove all paths in this graph from the path-list
find all paths that would conflict with the selected path and remove them from the path list (see Note). If a connection loses all of its alternative paths, re-expand its coarse graph - if this results in no new paths, the connection is deemed unroutable (see Section 5.4.3.3.1 for a discussion relating to failed connections).
update the cost of all affected paths
}

final expanded [illegible] discussed in the next section. The effect of the pruning algorithm is illustrated in Figure 5.12. The left half of the figure shows a fully expanded graph from an example circuit, while the corresponding pruned graph is on the right. Also shown are each graph's edges in the FPGA.

The choice to prune a vertex is based on the wire segment that corresponds to its incoming edge, as follows. For the special case of time-critical connections, the wire segments with the least delay are favored. For other connections, the wire segments that have thus far been included in the most other expanded graphs will be discarded. This helps the c_f cost function discover the wire segments that are in the least demand. Note that this introduces an order-dependence in the routing algorithm because the paths that are pruned from each expanded graph depend on the order in which the coarse graphs are expanded.

Note that when paths are discarded because of pruning, they are not necessarily abandoned permanently by the router. In phase 2, as CGE chooses connections, if routing conflicts consume all the alternatives for some graph, CGE re-invokes the graph expansion process to obtain a new set of paths if some exist.

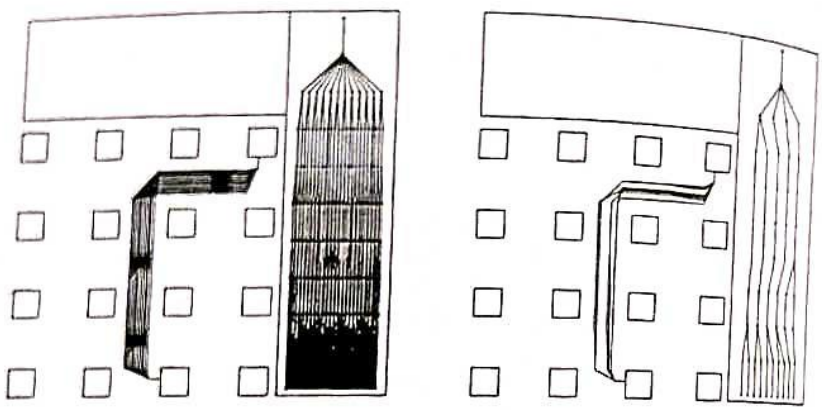


Figure 5.12 - *The Effect of Pruning.*

Circuit	#Blocks	#Conn	Source	Type
BUSC	109	392	UTD1	Bus Cntl
DMA	224	771	UTD2	DMA Cntl
BNRE	362	1257	BNR	Logic/Data
DFSM	401	1422	UTD1	State Mach.
Z03	586	2135	Zymos	8-bit Mult

Table 5.1 - *Experimental Circuits.*

❖ **What are the routing results**

5.4.3.5.2 Routing Results

The familiar yardstick of channel density is used as a measure of the quality of the detailed router. The 'Channel density' column in Table 5.2 shows the maximum channel density over all channels for each circuit. This represents a lower bound on the number of tracks per routing channel that is needed for each example. The real track requirements will depend on the flexibility of the routing architecture, because the channel density measure does not consider the amount of connectivity that is available in the routing structures. The maximum flexibility has $F_s = 3W$ and $F_c = W$, where there are W tracks per channel. For the results in Table 5.2, the FPGA parameters are based on the Xilinx 3000 series [Xili89] ($F_s = 6$, $F_c = 0.6W$). Table 5.2 gives the minimum number of tracks per channel that CGE needs in order to route 100 percent of the connections. The values for W are slightly greater than the global router minimum, which are excellent results considering the low flexibility of the FPGA routing architecture. Note that, although not shown, if F_c is increased to 0.8W, CGE achieves the absolute minimum number of tracks for all the circuits.

For comparison purposes, the same problems have also been routed using CGE with its c_f cost facility disabled. In this mode CGE has no ability to resolve routing conflicts and is thus a sequential router, similar to a maze router. At first glance, this may seem to be an unrealistic comparison because some maze routers are guided by cost functions that aid in finding good routes for connections. However, the 'maze' router used here has, in effect, access to the cost function that was used to solve the global routing, which is based on balancing the densities of all routing channels. Notwithstanding, this is a constrained 'maze' router because it is confined to remain within the global route of each connection, and the comparisons are valid

Circuit	Channel density	W required by CGE	W for 'maze'
BUSC	9	10	15
DMA	10	10	15
BNRE	11	12	20
DFSM	10	10	18
Z03	11	13	18

Table 5.2 - *CGE Minimum W for 100 % routing* ($F_c = 0.6W$, $F_s = 6$).

only in that context. The rightmost column in Table 5.2 gives the number of tracks that the 'maze' router requires to achieve 100 percent routing. These results demonstrate that the 'maze' router needs an average of 60 percent more tracks than CGE. This shows that resolving routing conflicts is important and that CGE addresses this issue well. Figure 5.15 presents the detailed routing for circuit BUSC, with the FPGA parameters in Table 5.2; the logic blocks are shown as solid boxes, whereas the S and C blocks are dashed boxes.

Name of net	# of switches without critical processing	# of switches with critical processing
#143	15	5
#144	14	4
#220	10	3
#280	15	2
#351	15	4

Table 5.3 - *Critical Connection Routing Delay Optimization.*